

Object Oriented Programming (Java)

Complete Sessional Study Notes

Nirma University | MCA Semester I | Built from your sessional slides + 4 years of final-exam papers

How to use these notes

- Every topic follows the same pattern: **plain-English idea, analogy, example/code with output, diagram, exam tip.**
- Topics are tagged with the priority I found in the past-paper analysis. Study the red tags first.
- Every code sample is short enough to type from memory. In the exam, **definition, comparison table, example program** is the winning structure.
- Boxes marked **MODEL ANSWER** are ready-to-write exam answers.

Tag	Meaning	What to do
[🔥 HIGH-YIELD]	Appeared in 3 or 4 of 4 past papers (or 8 marks every time)	Master fully. Be able to write it from memory.
[★ ESSENTIAL FOR EXAM]	Appeared 1 to 3 times, or is the foundation for a high-yield topic	Learn the definition, one example and one difference.
[📘 GOOD TO KNOW]	Not yet asked in finals, but in your slides (could be MCQs or short answers)	Skim once. Know the one-line idea and one example.

Contents and priority map

Unit	Topics	Key exam items	Priority
1	Java basics, JDK / JRE / JVM, execution flow, features, setup	JDK vs JRE vs JVM, write once run anywhere	🔥 HIGH
2	Tokens, keywords, identifiers, variables, data types	Define tokens and types of tokens	★ MEDIUM
3	Decision making: if, else-if ladder, switch, break, continue	Else-if ladder program, switch example	★ MEDIUM
4	Operators and type conversion	Relational operators with example, output tracing	★ MEDIUM
5	Loops (while, for, do-while, nested) and arrays	while vs do-while, pattern programs, loop tracing	🔥 HIGH
6	String handling	equals, concat, compareTo, substring, contains, replace, length	🔥 HIGH
7	Classes, objects and methods	Employee class program (gross salary)	★ MEDIUM
8	Constructors	Constructor vs method (every paper!)	🔥 HIGH
9	Static variable, static block, static method	Static vs non-static, program	🔥 HIGH
10	Inheritance	Multilevel and hierarchical programs	★ MEDIUM

Unit	Topics	Key exam items	Priority
11	Revision kit: cheat sheets, practice questions with answers, mistakes	Last-day revision	★ USE LAST

 REMEMBER

Suggested study order (highest return first): Unit 8 (Constructors), Unit 1 (JDK/JRE/JVM), Unit 5 (Loops), Unit 6 (Strings), Unit 9 (Static), then Units 4, 3, 10, 7, 2.

Self-study flag: `String.contains()` and `String.replace()` were asked in finals but are not in your slides. They are fully covered in Unit 6 here.

Unit 1: Java Fundamentals [🔥 HIGH-YIELD: JDK / JRE / JVM asked in 4 of 4 papers]

1.1 What is Java? [📖 GOOD TO KNOW: foundation]

Java is a **high-level, general-purpose, object-oriented** programming language created by **James Gosling and Patrick Naughton at Sun Microsystems in 1991**. It is now owned by **Oracle**. It was designed to be small, simple and portable across platforms and operating systems. You can build enterprise, web, mobile and cloud applications with it.

💡 ANALOGY

Think of a **PDF file**. It looks the same on a phone, a Windows laptop or a Mac, because every device has a PDF reader. Java works the same way: your compiled program looks the same everywhere, because every device has a **JVM** (the "Java reader").

Edition	Full name	Used for
Java SE	Standard Edition	Core libraries to build basic Java applications. This is what you learn in this course.
Java EE	Enterprise Edition	Libraries for large distributed (web, server) applications. It depends on Java SE.

1.2 JDK vs JRE vs JVM [🔥 HIGH-YIELD: asked 4/4 papers, 6 to 8 marks]

These three always appear together. The easiest way to remember them is that **each one sits inside the next**: the JVM is inside the JRE, and the JRE is inside the JDK.

 **Figure: JDK contains JRE, JRE contains JVM**

```

+- JDK -----+
| Development tools: javac, java, javadoc, jar, jdb |
| +- JRE -----+
| | Class libraries (java.lang, java.util ...) |
| | +- JVM -----+
| | | Executes bytecode (.class) |
| | | Memory management, garbage collection |
| | +-----+
| +-----+
+-----+
  
```

💡 ANALOGY

Think of a **kitchen**:

- **JVM** = the stove (it actually cooks, i.e. runs your program).
- **JRE** = stove + ingredients and utensils (everything needed to **serve** a ready recipe).
- **JDK** = whole kitchen + recipe-writing tools (everything needed to **create** new recipes, i.e. write, compile and run programs).

Point	JDK	JRE	JVM
Full form	Java Development Kit	Java Runtime Environment	Java Virtual Machine
Purpose	To develop (write, compile, debug) Java programs	To run Java programs	To execute bytecode line by line
Contains	Development tools (javac, java, javadoc) + JRE	JVM + core class libraries	Class loader, bytecode verifier, execution engine (interpreter + JIT), garbage collector

Point	JDK	JRE	JVM
Who needs it	Programmers	End users who only run programs	Part of the JRE. Always working in the background
Can compile code?	Yes (has javac)	No	No
Platform dependent?	Yes (separate download per OS)	Yes	Yes (one JVM per OS), but bytecode is platform independent

Components of the JDK (asked in Jan-2025)

Tool	What it does
<code>javac</code>	The compiler . Converts <code>.java</code> source code into <code>.class</code> bytecode.
<code>java</code>	The launcher/interpreter . Starts the JVM and runs the <code>.class</code> file.
<code>javadoc</code>	Generates HTML documentation from comments in the code.
<code>jar</code>	Packages many class files into one <code>.jar</code> archive.
<code>jdb</code>	The command-line debugger .
<code>javap</code>	Disassembler: shows what is inside a <code>.class</code> file.
JRE	Bundled inside the JDK so the program can run after compiling.

How a Java program runs (execution process)

Figure: From source code to output

```

[ Hello.java ]      <-- you write this (human readable)
    |
    |  javac (compiler: checks syntax, creates bytecode)
    v
[ Hello.class ]     <-- BYTECODE (same on every computer)
    |
    |  JVM (class loader -> verifier -> interpreter / JIT)
    v
[ Machine code ]   -->  OUTPUT on screen

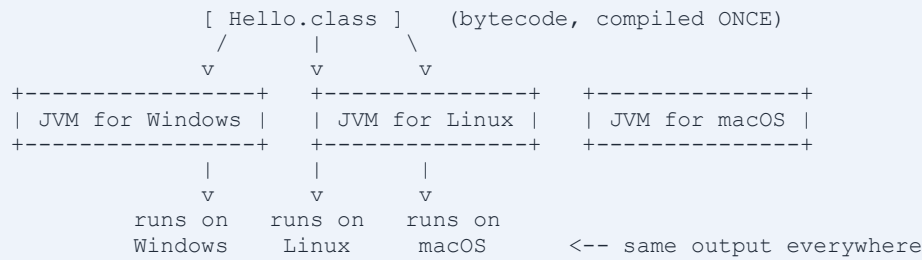
```

1. You write the program in a text editor and **save it as `ClassName.java`**.
2. `javac ClassName.java` compiles it. If there are no errors, `ClassName.class` is created.
3. `java ClassName` starts the JVM. The JVM loads the class, verifies the bytecode and executes it.

1.3 Write Once, Run Anywhere (WORA) [HIGH-YIELD: asked in Dec-2025]

The compiler does **not** produce machine code for one specific computer. It produces **bytecode**, a neutral in-between language. Each operating system has its own JVM that translates this bytecode into that machine's native instructions.

 **Figure: One bytecode file, many JVMs**



MODEL ANSWER (write this in the exam)

Importance of JVM: The JVM is the engine that runs Java bytecode. It loads classes, verifies bytecode for security, executes it (interpreter + JIT compiler) and manages memory using garbage collection.

Why it gives "write once, run anywhere": javac converts source code to platform-independent **bytecode**. The same **.class** file can be taken to any machine that has a JVM, because the JVM for that platform converts the bytecode to native instructions. So the programmer compiles once and does not rewrite the program for each operating system.

1.4 Features of Java [**GOOD TO KNOW:** asked once as "features of OOP/Java"]

Feature	Meaning
Simple	Easy syntax, inspired by C++ but removes confusing parts such as pointers.
Object-oriented	Everything is modelled with classes and objects (encapsulation, inheritance, polymorphism, abstraction).
Platform independent	Bytecode runs on any OS with a JVM (WORA).
Secure	No explicit pointers, bytecode verifier and security manager.
Robust	Strong memory management (garbage collection) and exception handling.
Portable	Same data sizes on every platform (an <code>int</code> is always 4 bytes).
Multithreaded	Can do many tasks at the same time using threads.
Distributed	Built for network applications.
High performance	JIT (Just-In-Time) compiler speeds up bytecode execution.

EXAM TIP

The four **OOP pillars** are often asked together with this topic: **Encapsulation** (wrap data and methods in a class), **Inheritance** (child gets parent's properties), **Polymorphism** (one name, many forms), **Abstraction** (hide details, show essentials). Your sessional covers inheritance in Unit 10.

1.5 Setting up and running your first program [**GOOD TO KNOW:** practical / viva]

A. Setting the Java path (Windows)

The OS must know where `javac` lives. We store the JDK location in an environment variable called **JAVA_HOME**.

1. Open Command Prompt and copy the JDK folder path.
2. `set JAVA_HOME=<your JDK path>` then verify with `echo %JAVA_HOME%`.
3. `set PATH=%JAVA_HOME%\bin` then verify with `echo %PATH%`.

- Alternative: search "env" in Windows, open **Edit the system environment variables, Advanced tab, Environment Variables**, and add the same values.

B. First program from the console

HelloWorldApp.java (file name **MUST** match the public class name)

```
public class HelloWorldApp {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Commands

```
C:\> javac HelloWorldApp.java      (creates HelloWorldApp.class)
C:\> java HelloWorldApp            (runs it)
```

Output:

```
Hello, World!
```

REMEMBER

- `public static void main(String[] args)` is the **entry point**. The JVM looks for exactly this line to start your program.
- `System.out.println()` prints text and moves to a new line. `print()` stays on the same line.
- Best practice: **class names** start with a capital letter (`Employee`), **method and variable names** start lowercase in camelCase (`getName()`).

Eclipse IDE steps: File > New > Java Project > give a name > Finish. Right-click `src` > New > Class > name it (e.g. `Main`) > write code > Right-click project > Run As > Java Application.

Unit 2: Tokens, Variables and Data Types [★ ESSENTIAL FOR EXAM: Tokens asked in Jan-2023]

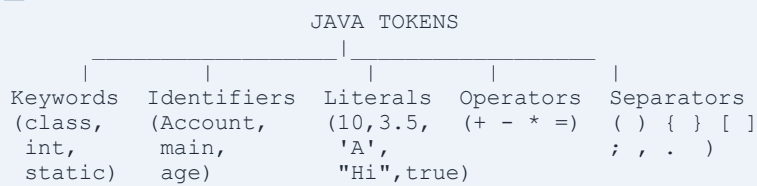
2.1 Java Tokens [★ ESSENTIAL FOR EXAM: asked Jan-2023, 6 marks]

When you compile a program, the compiler breaks your code into the smallest meaningful pieces. Each piece is a **token**. A token is the smallest individual unit of a Java program.

💡 ANALOGY

A sentence like `"I love Java"` is made of words. Java code is made of tokens in the same way.

 **Figure: The 5 types of Java tokens**



Worked example: break a line into tokens

```
int age = 20;
```

Token	Type	Why
<code>int</code>	Keyword	Reserved word with a fixed meaning
<code>age</code>	Identifier	Name chosen by the programmer
<code>=</code>	Operator	Assignment operator
<code>20</code>	Literal	A fixed integer value
<code>;</code>	Separator	Ends the statement

1) Keywords

Reserved words with a predefined meaning. You cannot use them as names. Java has about **50 keywords**.

Examples: `class`, `public`, `static`, `void`, `int`, `for`, `if`, `else`, `while`, `new`, `return`, `this`, `super`, `extends`, `break`, `continue`, `switch`, `case`, `default`, `final`.

2) Identifiers

Names given by the programmer to classes, methods, variables, interfaces. Example: `Account`, `main`, `totalPrice`.

Rule	Valid vs invalid example
Can contain letters, digits, \$ and _	<code>total_1</code> , <code>\$cost</code> , <code>_x</code> are valid
Must not start with a digit	<code>2name</code> is invalid, <code>name2</code> is valid
No spaces	<code>my name</code> is invalid, <code>myName</code> is valid
Cannot be a keyword	<code>int</code> , <code>class</code> are invalid
Case-sensitive	<code>Age</code> and <code>age</code> are two different names
No length limit	Any length allowed

3) Literals

Fixed values written directly in code. Five types (from your slide):

Literal type	Examples	Note
Integer	10, -5, 100L	Whole numbers. <code>L</code> suffix = long
Floating point	3.14, 2.5f	<code>f</code> suffix = float, default is double
Character	'A', '7'	Single quotes, ONE character
String	"Hello"	Double quotes, text
Boolean	true, false	Only these two values

4) Operators and 5) Separators

Operators perform operations (+ - * / = == && ...), covered in Unit 4). **Separators** organise the code: () parentheses, { } braces (blocks), [] brackets (arrays), ; semicolon (end of statement), , comma, . dot.

MODEL ANSWER (write this in the exam)

A **token** is the smallest individual unit of a Java program that the compiler recognises. Java has five types of tokens: **keywords** (reserved words like `class`), **identifiers** (programmer-defined names like `age`), **literals** (fixed values like `10`, `"Hi"`), **operators** (symbols like `+`, `=`) and **separators** (symbols like `;`, `{`, `}`). Example: `int age = 20;` has 5 tokens.

2.2 Variables GOOD TO KNOW: foundation for everything

A **variable** is a **named memory location** that holds a value which can change while the program runs.

ANALOGY

A variable is a **labelled box**. The label is the name (`age`), the type decides what can go in the box (only numbers, only text...), and the content is the value (`20`). You can replace the content but the label stays.

Figure: A variable in memory

```
int age = 20;

+-----+
|  20   | <-- value stored
+-----+
age      <-- name (label of the box)
(int)    <-- type (what kind of value fits)
```

```
int age;           // declaration
age = 20;          // initialization (assign value)
int marks = 85;    // declare + initialize together
marks = 90;        // value can change
double price = 99.5, tax = 5.0; // many of same type
```

REMEMBER

Variable names: nouns, start with a **lowercase** letter, camelCase for multiple words (`totalPrice`, `studentName`).

2.3 Data types [GOOD TO KNOW: foundation]

A **data type** tells Java what kind of data a variable stores and how much memory to reserve. There are two groups: **primitive** (8 built-in basic types) and **non-primitive** (classes, arrays, interfaces, Strings).

ANALOGY

Your slide uses a **coffee shop**: small, medium, large cups hold different amounts. `byte`, `short`, `int`, `long` are like cups of growing size. Pour a big drink into a small cup and it overflows!

Type	Size	Range / meaning	Default	Example
<code>byte</code>	1 byte	-128 to 127	0	<code>byte b = 100;</code>
<code>short</code>	2 bytes	-32,768 to 32,767	0	<code>short s = 5000;</code>
<code>int</code>	4 bytes	about -2.1 billion to 2.1 billion	0	<code>int age = 25;</code>
<code>long</code>	8 bytes	very large whole numbers	0L	<code>long pop = 8000000000L;</code>
<code>float</code>	4 bytes	decimal, about 7 digits precision	0.0f	<code>float pi = 3.14f;</code>
<code>double</code>	8 bytes	decimal, about 15 digits precision	0.0d	<code>double d = 3.14159;</code>
<code>char</code>	2 bytes	a single character (Unicode)	'\u0000'	<code>char grade = 'A';</code>
<code>boolean</code>	1 bit*	<code>true</code> or <code>false</code>	<code>false</code>	<code>boolean pass = true;</code>

Point	Primitive	Non-primitive (reference)
Defined by	Java itself (8 types)	Programmer / library (classes, arrays, interfaces)
Stores	The actual value	The address (reference) of an object
Default value	0, false ...	<code>null</code>
Example	<code>int x = 5;</code>	<code>String name = "Asha";, int[] arr</code>
Start with	lowercase (<code>int</code>)	Uppercase by convention (<code>String</code>)

COMMON MISTAKE

"A" (double quotes) is a **String**; 'A' (single quotes) is a **char**. `float f = 3.14;` gives an error because 3.14 is a double. Write `3.14f`.

Unit 3: Decision Making [★ ESSENTIAL FOR EXAM: else-if & switch asked in finals]

Decision-making statements let a program **choose a path** depending on a condition, just as you decide "if it rains, take an umbrella".

3.1 if statement [📌 GOOD TO KNOW: building block]

```
int marks = 75;
if (marks >= 40) {
    System.out.println("Pass");
}
```

Output:

Pass

The block runs **only if** the condition is **true**. Otherwise it is skipped.

3.2 if...else statement [📌 GOOD TO KNOW: building block]

```
int marks = 30;
if (marks >= 40) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
```

Output:

Fail

Exactly **one of two** blocks always runs.

3.3 Nested if [📌 GOOD TO KNOW: building block]

An **if** placed inside another **if**. The inner one is checked only when the outer one is true.

```
int age = 20; boolean hasLicense = true;
if (age >= 18) {
    if (hasLicense) {
        System.out.println("You can drive");
    } else {
        System.out.println("Get a license first");
    }
}
```

Output:

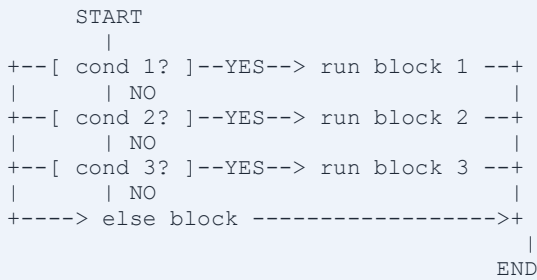
You can drive

3.4 else-if ladder [★ ESSENTIAL FOR EXAM: asked Dec-2025, 8 marks]

When there are **many** possible outcomes, chain conditions one below another. Java checks from the top and runs the **first** block whose condition is true, then skips the rest.

💡 ANALOGY

An else-if ladder is like a **security queue with several counters**: you try counter 1, if it does not suit you move to counter 2, and so on. The moment one accepts you, you stop moving.


Figure: else-if ladder flow

```

START
|
+--[ cond 1? ]--YES--> run block 1 --+
|      | NO
+--[ cond 2? ]--YES--> run block 2 --+
|      | NO
+--[ cond 3? ]--YES--> run block 3 --+
|      | NO
+----> else block ----->+
|
END

```

Real-life program: electricity-bill slab / grade calculator

```

public class GradeDemo {
    public static void main(String[] args) {
        int marks = 72;
        if (marks >= 90) {
            System.out.println("Grade A+");
        } else if (marks >= 75) {
            System.out.println("Grade A");
        } else if (marks >= 60) {
            System.out.println("Grade B");
        } else if (marks >= 40) {
            System.out.println("Grade C");
        } else {
            System.out.println("Fail");
        }
    }
}

```

Output:

Grade B

Trace: $72 \geq 90$ false, $72 \geq 75$ false, $72 \geq 60$ **true**, so it prints Grade B and skips the remaining checks.

Point	Simple if-else	else-if ladder
Number of outcomes	Only 2 (true / false)	3 or more
Conditions	One condition	Multiple conditions checked in order
Execution	One of two blocks	Only the first true block, rest skipped
Real-life use	Pass / Fail	Grades A / B / C / Fail, tax slabs, discounts

EXAM TIP

In the exam: define, draw the flow (above), write the grade program, then add the difference table. That is a full 8 marks.

3.5 switch statement [★ ESSENTIAL FOR EXAM: asked Jan-2023, 6 marks]

switch selects **one of many blocks** by matching a single value against **case** labels. It is cleaner than a long else-if ladder when you compare **one variable with fixed values** (days, menu choices).

```
public class SwitchDemo {
    public static void main(String[] args) {
        int day = 3;
        switch (day) {
            case 1: System.out.println("Monday");    break;
            case 2: System.out.println("Tuesday");   break;
            case 3: System.out.println("Wednesday"); break;
            case 4: System.out.println("Thursday");  break;
            default: System.out.println("Other day");
        }
    }
}
```

Output:

Wednesday

 REMEMBER

- The expression can be `byte`, `short`, `int`, `char`, `String` or an `enum`. **Not** `long`, `float`, `double` or `boolean`.
- `break` stops the switch. **Without** `break`, execution "falls through" into the next cases.
- `default` runs when no case matches (optional, usually last).
- Case values must be **constants** and **unique**.

Fall-through demo (a favourite output question)

```
int x = 2;
switch (x) {
    case 1: System.out.println("A");
    case 2: System.out.println("B");
    case 3: System.out.println("C");
    default: System.out.println("D");
}
```

Output:B
C
D

Match found at `case 2`, and with no `break` it keeps running every line below it.

Point	switch	else-if ladder
Compares	One value against constants (equality only)	Any condition (<code>></code> , <code><</code> , <code>&&</code> , ranges)
Readability	Better for many fixed choices	Better for ranges
Needs break	Yes, to avoid fall-through	No

3.6 break and continue [**GOOD TO KNOW:** appeared in output tracing, Dec-2025]

Statement	What it does	Effect
<code>break</code>	Exits the loop (or switch) immediately	Stops everything
<code>continue</code>	Skips the rest of the current iteration	Jumps to the next iteration

break

```
for (int i = 1; i <= 5; i++) {  
    if (i == 3) break;  
    System.out.println(i);  
}  
// prints 1, 2 then loop ends
```

Output:

```
1  
2
```

continue (Dec-2025 Q1A part 2)

```
for (int i = 1; i <= 10; i++) {  
    if (i == 7) continue;  
    System.out.println(i);  
}
```

Output:

```
1 2 3 4 5 6 8 9 10    (each on a new line; 7 is skipped)
```

 ANALOGY

break = leaving the cinema in the middle of the movie. **continue** = skipping one boring scene and watching the next.

Unit 4: Operators and Type Conversion [★ ESSENTIAL FOR EXAM: Relational operators asked in 3 of 4 papers]

An **operator** is a symbol that performs an operation on **operands** (values). In `a + b`, `+` is the operator and `a, b` are operands.

4.1 Arithmetic operators [📌 GOOD TO KNOW: foundation]

Operator	Meaning	Example (a=10, b=3)
<code>+</code>	Addition	<code>a + b = 13</code>
<code>-</code>	Subtraction	<code>a - b = 7</code>
<code>*</code>	Multiplication	<code>a * b = 30</code>
<code>/</code>	Division	<code>a / b = 3</code> (integer division drops decimals)
<code>%</code>	Modulus (remainder)	<code>a % b = 1</code>
<code>++ / --</code>	Increment / decrement by 1	<code>a++</code> makes <code>a = 11</code>

⚠️ COMMON MISTAKE

`7 / 2` is **3**, not 3.5, because both are integers. Use `7 / 2.0` to get 3.5.

Pre vs post increment

```
int x = 5;
int a = x++; // post: use x (5) first, THEN increase -> a=5, x=6
int y = 5;
int b = ++y; // pre: increase first, THEN use -> b=6, y=6
System.out.println(a + " " + x + " " + b + " " + y);
```

Output:

5 6 6 6

4.2 Relational operators [★ ESSENTIAL FOR EXAM: asked Feb-2022, Jan-2023 (6 to 8 marks)]

Relational operators **compare two values** and always give a **boolean result** (`true` or `false`). They are used inside `if`, `while` and `for` conditions.

Operator	Meaning	Example (a=10, b=20)	Result
<code>==</code>	Equal to	<code>a == b</code>	false
<code>!=</code>	Not equal to	<code>a != b</code>	true
<code>></code>	Greater than	<code>a > b</code>	false
<code><</code>	Less than	<code>a < b</code>	true
<code>>=</code>	Greater than or equal to	<code>a >= 10</code>	true
<code><=</code>	Less than or equal to	<code>b <= 15</code>	false

```

public class RelationalDemo {
    public static void main(String[] args) {
        int a = 10, b = 20;
        System.out.println("a == b : " + (a == b));
        System.out.println("a != b : " + (a != b));
        System.out.println("a > b : " + (a > b));
        System.out.println("a < b : " + (a < b));
        System.out.println("a >= 10: " + (a >= 10));
        System.out.println("b <= 15: " + (b <= 15));
    }
}

```

Output:

```

a == b : false
a != b : true
a > b : false
a < b : true
a >= 10: true
b <= 15: false

```

⚠ COMMON MISTAKE

= assigns, == compares. Writing `if (a = 5)` is a compile error in Java. For Strings, use `.equals()` instead of `==` (see Unit 6).

4.3 Assignment operators [GOOD TO KNOW: foundation]

Operator	Same as	Example (x = 10)
=	assign	x = 10
+=	x = x + 5	x += 5 gives 15
-=	x = x - 5	x -= 5 gives 5
*=	x = x * 2	x *= 2 gives 20
/=	x = x / 2	x /= 2 gives 5
%=	x = x % 3	x %= 3 gives 1

4.4 Logical operators [ESSENTIAL FOR EXAM: used in Dec-2025 output-tracing question]

Logical operators **combine** boolean conditions.

Operator	Name	Rule
&&	AND	true only if both sides are true
	OR	true if at least one side is true
!	NOT	flips true to false and false to true

A	B	A && B	A B	!A
true	true	true	true	false
true	false	false	true	false
false	true	false	true	true

A	B	A && B	A B	!A
false	false	false	false	true

REMEMBER

Short-circuit: in `A && B`, if A is false Java does **not** even check B. In `A || B`, if A is true B is skipped.

Exam-style tracing (Dec-2025 Q1A part 3)

```
int x = 200, y = 50, z = 100;
if (x > y && y > z) { System.out.println("Hello"); }
if (z > y && z < x) { System.out.println("Java"); }
if ((y+200) > x && (y+150) < z) { System.out.println("Hello Java"); }
```

Condition	Evaluation
<code>x > y && y > z</code>	200 > 50 is true, but 50 > 100 is false , so false (skipped)
<code>z > y && z < x</code>	100 > 50 true and 100 < 200 true, so true , prints Java
<code>(y+200) > x && (y+150) < z</code>	250 > 200 true, but 200 < 100 is false , so false (skipped)

Output:

Java

4.5 Bitwise operators [GOOD TO KNOW: in your slides, not yet asked]

Bitwise operators work on the **binary bits** of numbers. Your slide uses a = 0 and b = 1 for the truth table.

a	b	a & b (AND)	a b (OR)	a ^ b (XOR)
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

```
int a = 5; // binary 0101
int b = 3; // binary 0011
System.out.println(a & b); // 0001 = 1
System.out.println(a | b); // 0111 = 7
System.out.println(a ^ b); // 0110 = 6
System.out.println(~a); // flips all bits = -6
System.out.println(a << 1); // shift left = 10 (multiply by 2)
System.out.println(a >> 1); // shift right = 2 (divide by 2)
```

Output:

1
7
6
-6
10
2

4.6 Operator precedence [GOOD TO KNOW: helps in tracing]

When an expression has many operators, Java follows an order (highest first):

Level	Operators	Example
1	() parentheses, ++ --	(2 + 3) * 4 = 20
2	* / %	2 + 3 * 4 = 14
3	+ -	
4	< > <= >=	
5	== !=	
6	&&	
7		
8	= += -= (assignment, lowest)	

EXAM TIP

When unsure, **use brackets**. They always win and make your intention obvious.

4.7 Type conversion and casting [GOOD TO KNOW: in your slides, not yet asked]

Converting a value from one data type to another.

A. Automatic (widening) conversion

A **smaller** type goes into a **larger** type automatically, with no data loss. Think of pouring a small cup into a big jug.

Figure: Widening direction (automatic)

```
byte --> short --> int --> long --> float --> double
                        ^
                        char
(small, safe) -----> (large, safe)
```

```
int x = 10;
double d = x;           // automatic: int -> double
System.out.println(d); // 10.0
```

Output:

10.0

B. Explicit (narrowing) conversion = casting

A **larger** type into a **smaller** type needs a manual **cast** with (type). Data may be lost.

```
double pi = 9.78;
int i = (int) pi;           // cast: decimal part is CUT OFF (not rounded)
System.out.println(i); // 9
```

Output:

9

 COMMON MISTAKE

`int i = 9.78;` is a compile error ("possible lossy conversion"). You must write the cast `(int)`. Casting **truncates**, it never rounds.

Unit 5: Loops and Arrays [💧 HIGH-YIELD: Loops appeared in 4 of 4 papers]

A **loop** repeats a block of code as long as a condition is true. Without loops, printing numbers 1 to 100 would need 100 `println` lines. With a loop it is 3 lines.

💡 ANALOGY

A loop is like **running laps on a track**: keep running while "laps done < 5" is true; when the condition becomes false, stop.

🔑 REMEMBER

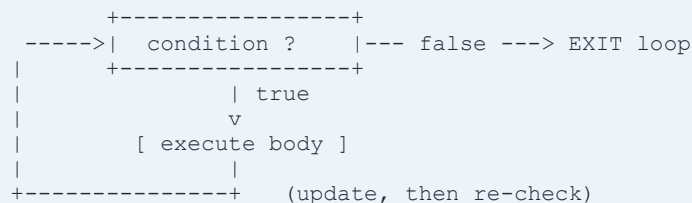
Every loop has **3 ingredients**: **(1) Initialization** (starting value), **(2) Condition** (keep going while true), **(3) Update** (change the counter, otherwise the loop never ends).

Loop	Type	Use when
<code>while</code>	Entry-controlled	You do not know how many times (e.g. until user enters 0)
<code>for</code>	Entry-controlled	You know the number of repetitions (1 to 10)
enhanced <code>for</code>	Entry-controlled	Visit every element of an array / collection
<code>do-while</code>	Exit-controlled	Body must run at least once (menus, input validation)

5.1 while loop [💧 HIGH-YIELD: while vs do-while asked Jan-2025]

Syntax: `while (condition) { body }`. The condition is checked **before** each iteration.

📊 Figure: while loop flow



```

int i = 1;                // 1. initialization
while (i <= 5) {           // 2. condition
    System.out.println("i = " + i);
    i++;                  // 3. update
}

```

Output:

```

i = 1
i = 2
i = 3
i = 4
i = 5

```

5.2 for loop [💧 HIGH-YIELD: asked Dec-2025 tracing, Feb-2022]

Syntax: `for (initialization; condition; update) { body }`. All three ingredients sit on **one line**, so it is compact and safe.

```
int sum = 0;
for (int i = 1; i <= 5; i++) {
    sum = sum + i;
}
System.out.println("Sum = " + sum);
```

Output:

Sum = 15

Pass	i	sum after the pass
1	1	1
2	2	3
3	3	6
4	4	10
5	5	15 (then i=6, condition false, exit)

Dec-2025 Q1A part 1: loop with a custom step

```
System.out.println("For Loop:");
for (int i = 55; i <= 100; i = i + 11) {
    System.out.println("i = " + i);
}
```

Pass	i	Condition i <= 100
1	55	true, prints
2	66	true, prints
3	77	true, prints
4	88	true, prints
5	99	true, prints
-	110	false, loop stops

Output:

```
For Loop:
i = 55
i = 66
i = 77
i = 88
i = 99
```

EXAM TIP

For any loop-tracing question, **draw a table** with one row per pass (as above). It prevents off-by-one errors and shows the examiner your justification.

5.3 Enhanced for loop (for-each) [GOOD TO KNOW: in your slides]

Syntax: `for (type var : array) { }`. No counter, no index. It picks each element automatically.

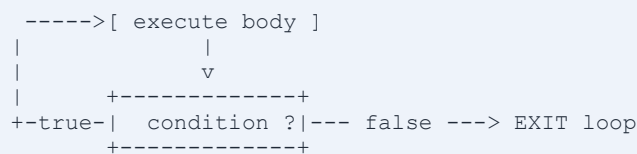
```
int[] marks = {70, 85, 90};
for (int m : marks) {
    System.out.println(m);
}
```

Output:

```
70
85
90
```

5.4 do-while loop [🔥 HIGH-YIELD: asked Jan-2025]

Syntax: `do { body } while (condition);` (note the **semicolon**). The body runs **first**, and the condition is checked **after**. So it always runs at least once.

 Figure: do-while loop flow
**do-while**

```
int i = 10;
do {
    System.out.println("Runs once, i = " + i);
    i++;
} while (i < 5);          // false from the start
```

Output:

```
Runs once, i = 10
```

same condition with while

```
int i = 10;
while (i < 5) {
    System.out.println("Never runs");
}
```

Output:

```
(nothing is printed)
```

Point	while	do-while
Condition checked	Before the body (entry-controlled)	After the body (exit-controlled)
Minimum runs	0 times (if condition is false at start)	At least 1 time
Semicolon	None after)	Required after <code>while(...)</code>
Syntax	<code>while(cond){ ... }</code>	<code>do{ ... }while(cond);</code>
Best for	Unknown count, may not run at all	Menus, "enter again" prompts, validation

👉 MODEL ANSWER (write this in the exam)

A **while** loop evaluates its condition before executing the body, so the body may never run. A **do-while** loop executes the body first and then evaluates the condition, so the body always runs at least once. [Add the two small examples above and the comparison table.]

5.5 Importance of loops [💧 HIGH-YIELD: asked Feb-2022 Q2B]

- **Avoid repetition:** write once, run many times.
- **Less code, fewer bugs,** and easy to change (change 5 to 500 in one place).
- **Process collections:** traverse arrays and lists.
- **Automation:** menus, validation, searching, calculating totals, printing patterns and tables.

5.6 Nested loops and pattern printing [💧 HIGH-YIELD: asked Jan-2023 Q3A, 8 marks]

A **nested loop** is a loop **inside** another loop. For every **one** pass of the outer loop, the inner loop runs **completely**.

💡 ANALOGY

A **clock**: the minute hand (inner loop) goes round all 60 minutes for every single step of the hour hand (outer loop). Or rows and columns in a classroom: the outer loop picks the **row**, the inner loop walks along the **seats**.

Use: printing patterns, multiplication tables, working with 2-D data (matrices, tables).

Example 1: star triangle

```
for (int i = 1; i <= 4; i++) {           // outer: rows
    for (int j = 1; j <= i; j++) {       // inner: stars in this row
        System.out.print("* ");
    }
    System.out.println();               // new line after each row
}
```

Output:

```
*
* *
* * *
* * * *
```

🔑 REMEMBER

Rule: **outer loop = rows, inner loop = columns**. `print()` keeps the cursor on the same line, and an empty `println()` after the inner loop moves to the next row.

Example 2: the exact Jan-2023 pattern

Print this using nested loops:

📊 Figure: Required output

```
1
0 1
1 0 1
0 1 0 1
1 0 1 0 1
```

Idea: row i has i numbers. Look at the position $(i + j)$: when it is **even** print 1, when it is **odd** print 0.
Check row 2: $j=1$ gives 3 (odd) so 0; $j=2$ gives 4 (even) so 1, which matches "0 1".

```
public class PatternDemo {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            for (int j = 1; j <= i; j++) {
                if ((i + j) % 2 == 0) {
                    System.out.print("1 ");
                } else {
                    System.out.print("0 ");
                }
            }
            System.out.println();
        }
    }
}
```

Output:

```
1
0 1
1 0 1
0 1 0 1
1 0 1 0 1
```

Row i	Inner j runs	Values of $(i+j)$ so printed
1	1	2 $\rightarrow$ 1
2	1 to 2	3, 4 $\rightarrow$ 0, 1
3	1 to 3	4, 5, 6 $\rightarrow$ 1, 0, 1
4	1 to 4	5, 6, 7, 8 $\rightarrow$ 0, 1, 0, 1
5	1 to 5	6, 7, 8, 9, 10 $\rightarrow$ 1, 0, 1, 0, 1

Example 3: multiplication table

```
for (int i = 1; i <= 3; i++) {           // table of 1, 2, 3
    for (int j = 1; j <= 3; j++) {
        System.out.print(i * j + "\t");
    }
    System.out.println();
}
```

Output:

```
1  2  3
2  4  6
3  6  9
```

⚠ COMMON MISTAKE

- **Infinite loop:** forgetting the update ($i++$) makes the loop run forever.
- **Off-by-one:** $i < 5$ runs 4 times from 1, but $i <= 5$ runs 5 times.
- **Stray semicolon:** `for (...);` ends the loop immediately and the block below runs only once.
- Using the **same variable name** i in both outer and inner loop is an error. Use i and j .

5.7 Arrays [GOOD TO KNOW: in your slides, not yet asked]

An **array** stores **many values of the same type** in one variable, in consecutive memory slots, each reached by an **index starting at 0**.

ANALOGY

An array is a **row of lockers**: one building (the array name), numbered doors (index 0, 1, 2 ...), and each locker holds one item of the same kind.

 **Figure: `int[] marks = {70, 85, 90, 65, 80};`**

index :	0	1	2	3	4
marks	70	85	90	65	80

marks[0] = 70 marks[4] = 80 marks.length = 5
 LAST valid index = length - 1 = 4

```
int[] a = new int[5];           // declare + create (default values 0)
int[] marks = {70, 85, 90, 65, 80}; // declare + initialize together
int[] b; b = new int[3];        // two-step form
System.out.println(marks[2]);   // 90
System.out.println(marks.length); // 5 (length is a property, no brackets)
```

Program: average of array elements

```
public class ArrayDemo {
    public static void main(String[] args) {
        int[] marks = {70, 85, 90, 65, 80};
        int sum = 0;
        for (int i = 0; i < marks.length; i++) {
            sum += marks[i];
        }
        double avg = (double) sum / marks.length;
        System.out.println("Sum = " + sum);
        System.out.println("Average = " + avg);
    }
}
```

Output:

```
Sum = 390
Average = 78.0
```

COMMON MISTAKE

Accessing `marks[5]` (or any index outside 0 to 4) throws **ArrayIndexOutOfBoundsException**. Remember: valid indexes are **0 to length-1**.

Unit 6: String Handling [🔥 HIGH-YIELD: asked in 3 of the last 3 papers]

A **String** is a sequence of characters, such as "Hello Java". In Java, String is **not** a primitive type. It is a **class** in `java.lang`, so every String is an object and comes with many ready-made **methods**.

💡 ANALOGY

A String is like a **printed sentence**. You can read it, measure it, copy part of it, but you cannot erase a letter on the original paper. To "change" it you must print a new sentence. That is **immutability**.

6.1 Creating Strings [📌 GOOD TO KNOW: foundation]

```
String s1 = "Java";           // string literal (goes in String Pool)
String s2 = "Java";           // reuses the SAME pooled object
String s3 = new String("Java"); // new object in heap (separate)
```

📊 Figure: String pool

STRING POOL (heap)		HEAP (normal)
+-----+		+-----+
"Java" <--- s1, s2		"Java" <--- s3
+-----+		+-----+

s1 and s2 point to the SAME object; s3 points to a DIFFERENT one

🔑 REMEMBER

Strings are immutable: once created, the content cannot change. Methods like `concat`, `toUpperCase`, `replace` return a **new String** and leave the original untouched.

```
String s = "Java";
s.concat(" Fun");           // result thrown away!
System.out.println(s);      // Java (unchanged)
s = s.concat(" Fun");        // store the new String back
System.out.println(s);      // Java Fun
```

Output:

```
Java
Java Fun
```

6.2 Index map used in all examples below [📌 GOOD TO KNOW: learn this picture]

```
String s = "Welcome to Java";
```

📊 Figure: Positions (index starts at 0; the space counts as a character)

char :	W	e	l	c	o	m	e		t	o		J	a	v	a
index:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

`length() = 15` (last index is 14)

6.3 The important methods [🔥 HIGH-YIELD: one named trio appears in every paper]

Quick lookup table, then one example each.

Method	Purpose	Returns	Example (s = "Welcome to Java")
<code>length()</code>	Number of characters	int	<code>s.length() = 15</code>
<code>charAt(i)</code>	Character at index i	char	<code>s.charAt(0) = W</code>
<code>substring(b)</code>	From index b to the end	String	<code>s.substring(11) = "Java"</code>
<code>substring(b, e)</code>	From b up to e-1	String	<code>s.substring(0, 7) = "Welcome"</code>
<code>indexOf(x)</code>	First position of x (-1 if absent)	int	<code>s.indexOf("Java") = 11</code>
<code>contains(x)</code>	Is x inside the string?	boolean	<code>s.contains("Java") = true</code>
<code>replace(a, b)</code>	Replace every a with b	String	<code>s.replace("Java", "World")</code>
<code>equals(x)</code>	Content equal? (case-sensitive)	boolean	<code>"java".equals("Java") = false</code>
<code>equalsIgnoreCase(x)</code>	Equal ignoring case	boolean	<code>"java".equalsIgnoreCase("Java") = true</code>
<code>compareTo(x)</code>	Dictionary order comparison	int	<code>"apple".compareTo("banana") = -1</code>
<code>concat(x)</code>	Joins two strings	String	<code>"Hi".concat(" there") = "Hi there"</code>
<code>toUpperCase()</code> / <code>toLowerCase()</code>	Change case	String	<code>"ab".toUpperCase() = "AB"</code>
<code>trim()</code>	Removes spaces at both ends	String	<code>" hi ".trim() = "hi"</code>

length()

```
String s = "Welcome to Java";
System.out.println(s.length());
```

Output:
15

Counts **every** character including spaces. For arrays, `length` has **no brackets** (`arr.length`); for Strings, `length()` **has brackets**.

substring()

```
String s = "Welcome to Java";
System.out.println(s.substring(11)); // from index 11 to the end
System.out.println(s.substring(0, 7)); // index 0 up to 6 (end index EXCLUDED)
System.out.println(s.substring(8, 10)); // "to"
```

Output:
Java
Welcome
to

REMEMBER

`substring(begin, end)` includes `begin` but **excludes** `end`. Number of characters returned = `end - begin`.

equals() vs == (very common question)

```
String a = "Java";
String b = "Java";
String c = new String("Java");
System.out.println(a == b);           // true   (same pooled object)
System.out.println(a == c);           // false  (different objects)
System.out.println(a.equals(c));       // true   (same CONTENT)
System.out.println("java".equals("Java")); // false
System.out.println("java".equalsIgnoreCase("Java")); // true
```

Output:

```
true
false
true
false
true
```

REMEMBER

== compares **references (addresses)**. **equals()** compares **content**. For Strings, **always use `equals()`**.

compareTo()

Compares two Strings **letter by letter** using Unicode values. Result: **0** = equal, **negative** = first is smaller (comes earlier alphabetically), **positive** = first is greater.

```
System.out.println("apple".compareTo("banana")); // 'a' - 'b' = -1
System.out.println("banana".compareTo("apple")); // 'b' - 'a' = 1
System.out.println("apple".compareTo("apple")); // 0
System.out.println("apple".compareTo("apricot")); // first difference at index 2:
'p' (112) - 'r' (114) = -2
```

Output:

```
-1
1
0
-2
```

concat()

```
String first = "Hello";
String full = first.concat(" World");
System.out.println(full);
System.out.println(first + " World!"); // + also joins Strings
```

Output:

```
Hello World
Hello World!
```

contains() (self-study: not in your slides)

```
String s = "Welcome to Java";
System.out.println(s.contains("Java")); // true
System.out.println(s.contains("java")); // false (case-sensitive)
System.out.println(s.contains("Python")); // false
```

Output:

```
true
false
false
```

replace() (self-study: not in your slides)

```
String s = "Welcome to Java";
System.out.println(s.replace('o', '0'));           // every o becomes 0
System.out.println(s.replace("Java", "World"));    // replace a whole word
System.out.println(s);                             // original is unchanged!
```

Output:

```
Welc0me t0 Java
Welcome to World
Welcome to Java
```

Case, trim and charAt

```
String s = "  Java Class  ";
System.out.println(s.trim());           // "Java Class"
System.out.println(s.trim().toUpperCase()); // "JAVA CLASS"
System.out.println(s.trim().toLowerCase()); // "java class"
System.out.println("Java".charAt(2));    // 'v'
```

Output:

```
Java Class
JAVA CLASS
java class
v
```

6.4 Model answers for the three past-paper trios [💧 HIGH-YIELD: exact asked combinations]**📌 MODEL ANSWER (write this in the exam)****Dec-2025: replace(), length(), substring()**

- `replace(old, new)` returns a **new** String in which every occurrence of `old` is replaced by `new`.
"Java".replace('a', 'o') gives "Jovo".
- `length()` returns the **number of characters** as an `int`. "Java".length() gives 4.
- `substring(b, e)` returns the part from index `b` to `e-1`. "Java".substring(1, 3) gives "av".

Difference: replace **changes content** (returns modified copy), length **counts**, substring **extracts a part**.

📌 MODEL ANSWER (write this in the exam)**Jan-2025: compareTo(), contains(), substring()**

- `compareTo` returns 0 / negative / positive by dictionary order: "a".compareTo("b") = -1.
- `contains` returns **boolean**: "Java".contains("av") = true.
- `substring` as above.

 MODEL ANSWER (write this in the exam)**Jan-2023: equals(), concat(), substring()**

- `equals` checks content equality, `concat` joins strings, `substring` extracts a part. Give one line of code and output for each.

 EXAM TIP

A String question is usually 6 to 8 marks: **1 line on the String class** (immutable, in `java.lang`), then **each method: purpose, syntax, example, output**. Aim for 3 methods with full examples.

Unit 7: Classes, Objects and Methods [★ ESSENTIAL FOR EXAM: Employee-class program asked Dec-2025]

7.1 Class and object [★ ESSENTIAL FOR EXAM: foundation of OOP]

A **class** is a **blueprint** (template) that describes the data (variables) and behaviour (methods) of something. An **object** is a **real instance** created from that blueprint.

💡 ANALOGY

Class = **cookie cutter**. Object = each **cookie** made with it. One cutter, many cookies; each cookie has its own flavour (data) but the same shape (structure).

📊 Figure: One class, many objects

<pre> +- class Customer -----+ Variables (state): name totalPrice Methods (behaviour): payBill() display() +-----+ </pre>	<pre> +- c1 (object) -----+ name = "Ravi" totalPrice = 500 +-----+ +- c2 (object) -----+ name = "Meena" totalPrice = 250 +-----+ </pre>
--	--

```

class Customer {
    String name;
    double totalPrice;
    void display() {
        System.out.println(name + " : " + totalPrice);
    }
}

public class Main {
    public static void main(String[] args) {
        Customer c1 = new Customer(); // create object with 'new'
        c1.name = "Ravi"; c1.totalPrice = 500;
        Customer c2 = new Customer();
        c2.name = "Meena"; c2.totalPrice = 250;
        c1.display(); // object.method()
        c2.display();
    }
}

```

Output:

```

Ravi : 500.0
Meena : 250.0

```

7.2 Methods [★ ESSENTIAL FOR EXAM: used in every program]

A **method** is a named block of statements that represents a **behaviour** of a class. You write it once and call it many times (reusability).

Syntax

```

accessModifier returnType methodName(parameters) {
    // body
    return value; // only if returnType is not void
}

// example:
public double payBill(double totalPrice, double discountPercentage) { ... }

```

Type of method	Example signature	Meaning
No parameters, no return	<code>void display()</code>	Just does something
Parameters, no return	<code>void setName(String n)</code>	Takes input, returns nothing
Parameters + return value	<code>double payBill(double p, double d)</code>	Takes input, gives back a result
No parameters, returns value	<code>double getSalary()</code>	Gives a value back

```

class Billing {
    double payBill(double totalPrice, double discountPercentage) {
        double discount = totalPrice * discountPercentage / 100;
        return totalPrice - discount; // sends result back to the caller
    }
}
public class Main {
    public static void main(String[] args) {
        Billing b = new Billing();
        double amount = b.payBill(1000, 10); // arguments: 1000 and 10
        System.out.println("Pay: " + amount);
    }
}

```

Output:

Pay: 900.0

 REMEMBER

Parameter = variable in the method definition (`totalPrice`). **Argument** = actual value passed when calling (`1000`). A `void` method returns nothing. Any other return type **must** use `return`.

Parameter passing: pass-by-value

Passing	What happens
Primitive (int, double ...)	A copy of the value is passed. Changing it inside the method does not affect the original.
Object / array	A copy of the reference is passed. Both point to the same object, so changing the object's content inside the method does affect the original.

```

class Box { int v = 1; }
public class PassDemo {
    static void changePrimitive(int n) { n = n + 10; }
    static void changeObject(Box b) { b.v = 99; }
    public static void main(String[] args) {
        int x = 5;
        changePrimitive(x);
        System.out.println(x); // 5 (copy was changed, not x)
        Box box = new Box();
        changeObject(box);
        System.out.println(box.v); // 99 (same object was changed)
    }
}

```

Output:5
99

Local vs instance variables

Point	Instance variable	Local variable
Declared	Inside the class, outside methods	Inside a method / block
Lives	As long as the object lives	Only while the method runs
Default value	Yes (0, null, false)	None , must be initialized before use
Accessible	By all non-static methods	Only inside that method

7.3 Program: Employee class with Gross Salary [★ ESSENTIAL FOR EXAM: asked Dec-2025 Q3A, 8 marks]

Question: Create an `Employee` class with employee ID, name and salary. Include methods to display the details and calculate **Gross Salary = Salary + HRA + DA**. *(Assumption to state in the exam: HRA = 20% and DA = 10% of basic salary.)*

```
class Employee {
    int empId;
    String name;
    double salary;                // basic salary

    Employee(int empId, String name, double salary) {    // constructor
        this.empId = empId;
        this.name = name;
        this.salary = salary;
    }

    void display() {                // display details
        System.out.println("Employee ID : " + empId);
        System.out.println("Name      : " + name);
        System.out.println("Basic Salary : " + salary);
    }

    double calculateGrossSalary() {    // Gross = Salary + HRA + DA
        double hra = salary * 20 / 100; // assumed 20%
        double da  = salary * 10 / 100; // assumed 10%
        return salary + hra + da;
    }
}

public class EmployeeDemo {
    public static void main(String[] args) {
        Employee e = new Employee(101, "Ravi", 30000);
        e.display();
        System.out.println("Gross Salary : " + e.calculateGrossSalary());
    }
}
```

Output:

```
Employee ID : 101
Name       : Ravi
Basic Salary : 30000.0
Gross Salary : 39000.0
```

Check: HRA = 20% of 30000 = 6000, DA = 10% = 3000, Gross = 30000 + 6000 + 3000 = **39000**.

 EXAM TIP

The "assume suitable data" instruction in the paper means: **write your assumption clearly** (e.g. "Assume HRA = 20%, DA = 10%"). The examiner awards marks for it.

Unit 8: Constructors [🔥 HIGH-YIELD: Constructor vs Method asked in 4 of 4 papers]

🔗 EXAM TIP

If you learn only ONE topic, learn this one. "Constructor vs method" has appeared in Feb-2022, Jan-2023, Jan-2025 and Dec-2025, worth 7 to 8 marks each time.

8.1 What is a constructor? [🔥 HIGH-YIELD: definition asked every paper]

A **constructor** is a **special method** that is **automatically called when an object is created** (with `new`) and is used to **initialize the object's variables**.

💡 ANALOGY

Buying a new phone: when you switch it on for the first time it runs a **setup** (language, Wi-Fi, account). A constructor is that **first-time setup** for every new object. You do not ask for it. It happens by itself at birth.

🔗 REMEMBER

Three golden rules of a constructor:

- **Same name as the class.**
- **No return type** (not even `void`).
- **Called automatically** when you write `new ClassName (...)`.

📄 Figure: What happens when you write: `Student s = new Student("Asha", 21);`

```

Step 1:  new                --> memory is allocated for the object
                                     +-----+
                                     | name = null      | (default values)
                                     | age  = 0         |
                                     +-----+
Step 2:  Student(...)      --> constructor RUNS automatically
                                     name = "Asha"; age = 21;
Step 3:  reference         --> address of the object is stored in s
                                     s ----> [ name="Asha", age=21 ]

```

8.2 Types of constructors [★ ESSENTIAL FOR EXAM: in your slides]

A. Default constructor (given by the compiler)

If you write **no constructor at all**, Java silently adds an empty parameterless one so that `new Student()` works.

B. Parameterless constructor (written by you)

A constructor **with no arguments** that you define yourself. Useful to give **fixed starting values**.

C. Parameterized constructor

Accepts **arguments** so that **each object gets different values** at creation.

```

class Student {
    String name;
    int marks;

    Student() {                                // parameterless
        name = "Unknown";
        marks = 0;
    }
    Student(String name, int marks) {          // parameterized
        this.name = name;                      // this.name = object's variable
        this.marks = marks;                    // marks      = parameter
    }
}

```

⚠ COMMON MISTAKE

- If you write **any** constructor (e.g. only the parameterized one), the compiler does **not** add the default one. Then `new Student()` gives a **compile error** unless you also write a parameterless constructor.
- Writing `void Student() {...}` makes it an ordinary **method**, not a constructor!

Constructor overloading

A class can have **multiple constructors** with **different parameter lists**. Java picks the right one by looking at the arguments you pass. In the Student class above, `Student()` and `Student(String, int)` are overloaded.

The this keyword

`this` means "**the current object**". Use it when a parameter has the **same name** as an instance variable:
`this.name = name;` means "set *my* name to the parameter name".

8.3 Constructor vs Method [💧 HIGH-YIELD: asked Feb-22, Jan-23, Jan-25, Dec-25]

Point	Constructor	Method
Name	Must be same as the class name	Any valid name (usually a verb, e.g. <code>display</code>)
Return type	None (not even <code>void</code>)	Must have one (<code>int</code> , <code>String</code> , or <code>void</code>)
Purpose	Initializes the object when it is created	Performs an action / behaviour of the object
Invocation	Automatically by JVM at <code>new</code>	Explicitly by the programmer: <code>obj.method()</code>
How often	Once per object, at creation	Any number of times, any time
Default version	Compiler provides a default one if none written	No default method is provided
Inheritance	Not inherited by child classes	Inherited by child classes
Modifiers	Cannot be <code>static</code> , <code>final</code> or <code>abstract</code>	Can be static, final, abstract
return statement	Cannot return a value	Returns a value if return type is not void

8.4 Complete program using both [💧 HIGH-YIELD: asked Dec-2025: "use of both"]

```

class Student {
    String name;
    int marks;

    Student() {                                // constructor 1

```

```

        name = "Unknown";
        marks = 0;
        System.out.println("Default constructor called");
    }
    Student(String name, int marks) {           // constructor 2
        this.name = name;
        this.marks = marks;
        System.out.println("Parameterized constructor called");
    }
    void display() {                           // method (no return)
        System.out.println(name + " scored " + marks);
    }
    int addBonus(int bonus) {                  // method (returns value)
        marks = marks + bonus;
        return marks;
    }
}

public class ConstructorDemo {
    public static void main(String[] args) {
        Student s1 = new Student();           // constructor runs by itself
        Student s2 = new Student("Asha", 80); // constructor runs by itself
        s1.display();                          // method: we call it
        s2.display();
        s2.addBonus(5);                        // method: we call it
        s2.display();
    }
}

```

Output:

```

Default constructor called
Parameterized constructor called
Unknown scored 0
Asha scored 80
Asha scored 85

```

Notice the order: the two constructor messages appear **first and without us calling them**, then the methods run only when we call them.

 MODEL ANSWER (write this in the exam)

A **constructor** is a special member of a class used to initialize an object. It has the **same name as the class**, **no return type**, and is **invoked automatically** when an object is created using **new**. A **method** defines the behaviour of an object, can have **any name**, **must declare a return type** (or **void**), and is **called explicitly** on an object. [Then draw the table in 8.3 and the program in 8.4. Mention that constructors can be overloaded.]

Unit 9: Static Variable, Static Block and Static Method [🔥 HIGH-YIELD:

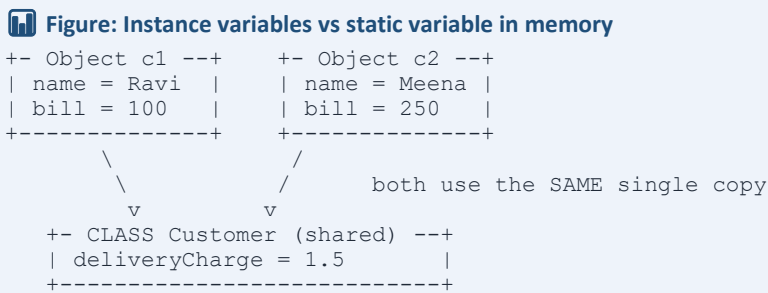
asked in 3 of 4 papers, 8 marks each]

9.1 Why do we need static? [🔥 HIGH-YIELD: concept behind every question]

Imagine a food-delivery app. Every customer has a **personal** name and bill, but the **delivery charge is the same for everyone**. If we store `deliveryCharge` inside every object, we waste memory and, worse, must update 1000 objects when the charge changes. We want **one shared copy**. That is what `static` gives us.

💡 ANALOGY

A classroom: each student has their **own notebook** (instance variable, one per object). The **whiteboard** is shared by everyone (static variable, one copy for the whole class). If the teacher writes on the whiteboard, all students see the change.


Figure: Instance variables vs static variable in memory

```

+- Object c1 ---+   +- Object c2 ---+
| name = Ravi |   | name = Meena |
| bill = 100  |   | bill = 250   |
+-----+       +-----+
      \         /
       v         v
+- CLASS Customer (shared) ---+
| deliveryCharge = 1.5      |
+-----+
  
```

both use the SAME single copy

9.2 Static variable [🔥 HIGH-YIELD: asked Jan-2023, Jan-2025]

- Declared with the keyword `static`. Only **one copy** exists, shared by **all objects** of the class.
- It is created **once, when the class is loaded**, not each time an object is made.
- Access it with the **class name**: `Customer.deliveryCharge` (better than `customer1.deliveryCharge`).
- Initialise at declaration: `private static float deliveryCharge = 1.5f;` or in a **static block**.

Counting objects with a static variable

```

class Counter {
    static int count = 0;        // ONE shared copy
    int id;                     // each object gets its own id
    Counter() {
        count++;                // shared counter goes up
        id = count;
    }
}

public class StaticVarDemo {
    public static void main(String[] args) {
        new Counter();
        new Counter();
        Counter c3 = new Counter();
        System.out.println("c3.id = " + c3.id);
        System.out.println("Total objects = " + Counter.count);
    }
}
  
```

Output:

```

c3.id = 3
Total objects = 3
  
```

9.3 Static block [★ ESSENTIAL FOR EXAM: in your slides]

A **static block** is used to initialise static variables when it cannot be done in a single line (e.g. needs calculation). It runs **once, when the class is loaded, before `main()`**.

```
class Demo {
    static int x;
    static {                                // static block
        x = 10;
        System.out.println("Static block executed");
    }
    public static void main(String[] args) {
        System.out.println("x = " + x);
    }
}
```

Output:

```
Static block executed
x = 10
```

9.4 Static method [💧 HIGH-YIELD: asked Dec-2025 Q2A, Jan-2023 Q5A]

A **static method** belongs to the **class**, not to any object. You can call it **without creating an object**: `ClassName.method()`. It is also the right tool to **get and set static variables** (because it needs no object).

💡 ANALOGY

A **bank's interest-rate notice board**. You can read it without opening an account (no object). But your **personal balance** is only visible if you have an account (needs an object).

🔑 REMEMBER

Rules of a static method:

- Can be called **without an object** (via class name).
- Can access **only static members directly**.
- **Cannot use `this`** (there is no current object).
- A **non-static** method can access **both** static and non-static members.
- `public static void main(...)` is static so the **JVM can start the program without creating an object**.

Complete program: declaration and invocation

```
class Customer {
    private String name;
    private double billAmount;
    private static float deliveryCharge = 1.5f;           // static variable

    Customer(String name, double billAmount) {
        this.name = name;
        this.billAmount = billAmount;
    }
    static float getDeliveryCharge() {                    // static getter
        return deliveryCharge;
    }
    static void setDeliveryCharge(float charge) {         // static setter
        deliveryCharge = charge;                         // 'this' NOT allowed here
    }
    void showBill() {                                     // non-static method
        System.out.println(name + " pays " + (billAmount + deliveryCharge));
    }
}
```

```

}

public class StaticDemo {
    public static void main(String[] args) {
        Customer c1 = new Customer("Ravi", 100);
        Customer c2 = new Customer("Meena", 250);
        c1.showBill();
        c2.showBill();

        System.out.println("Changing delivery charge...");
        Customer.setDeliveryCharge(2.0f);           // call WITHOUT an object
        c1.showBill();
        c2.showBill();
        System.out.println("Charge = " + Customer.getDeliveryCharge());
    }
}

```

Output:

```

Ravi pays 101.5
Meena pays 251.5
Changing delivery charge...
Ravi pays 102.0
Meena pays 252.0
Charge = 2.0

```

One call to `setDeliveryCharge` updated the charge for **both** customers, because they share one copy.

What goes wrong (compile errors)

```

class Test {
    int a = 5;                                // instance variable
    static void show() {
        System.out.println(a);                // ERROR: non-static variable a cannot be
                                                // referenced from a static context
        System.out.println(this.a);           // ERROR: 'this' not allowed in static method
    }
}
// Fix: create an object and use it: new Test().a

```

9.5 Static vs non-static [🔥 HIGH-YIELD: asked Dec-2025 (accessibility and usage)]

Point	Static method	Non-static (instance) method
Belongs to	The class	An object (instance)
Call using	<code>ClassName.method()</code> (no object needed)	<code>object.method()</code> (object required)
Accessibility	Can access only static variables and methods directly	Can access both static and non-static members
this keyword	Not allowed	Allowed
Memory	One copy per class	Works on each object's own data
Typical use	Utility / helper methods, <code>main()</code> , getters/setters for static data	Behaviour that depends on an object's data

Point	Static variable	Instance (non-static) variable
Copies	One shared by all objects	One per object
Created	Once, when class is loaded	Every time an object is created

Point	Static variable	Instance (non-static) variable
Access	Via class name (preferred)	Via object only
Example	Delivery charge, object counter, company name	Customer name, student marks

MODEL ANSWER (write this in the exam)

Role of static method: A static method belongs to the class, so it can be invoked directly with the class name without creating an object. It is used for utility operations and for reading or updating static variables (through static getter/setter methods). `main()` itself is static for this reason.

Static vs non-static in accessibility: a static method can directly access only static members and cannot use `this`; a non-static method can access both static and instance members.

[Then add the Customer program above or the Counter program.]

REMEMBER

Relationship between static variable and static method (Jan-2023): a static variable is shared data of the class, and static methods are the natural way to read or change it without needing an object. A static method can touch static variables but never instance variables.

Unit 10: Inheritance [★ ESSENTIAL FOR EXAM: multilevel & hierarchical asked in Feb-2022, Jan-2023]

Inheritance lets a new class (**child / subclass / derived class**) **reuse** the variables and methods of an existing class (**parent / superclass / base class**) using the keyword `extends`. It models an "IS-A" relationship: a `RegularCustomer` **is a** `Customer`.

💡 ANALOGY

You inherit your **surname and some traits** from your parents without anyone re-writing them, and you can add your own skills. Or: a **Car is a Vehicle**, so it already has wheels and an engine (from Vehicle) and adds its own features (AC, music system).

Syntax

```
class Parent { ... }
class Child extends Parent { ... } // Child gets everything public/protected from Parent
```

10.1 Types of inheritance [★ ESSENTIAL FOR EXAM: asked in Feb-2022 and Jan-2023]

 **Figure: The four types (A, B, C, D are classes; arrow = "extends")**

1) SINGLE

```

A
^
|
B
```

2) MULTILEVEL

```

A
^
|
B
^
|
C
```

3) HIERARCHICAL

```

  A
 ^ ^ ^
 | | |
B  C  D
```

4) MULTIPLE
(NOT in Java classes)

```

  A      B
   \    /
    v
   C
```

Type	Meaning	Example
Single	One class extends one class	<code>RegularCustomer</code> extends <code>Customer</code>
Multilevel	A chain: child becomes parent of another class	<code>Animal</code> > <code>Dog</code> > <code>Puppy</code>
Hierarchical	Many classes extend one parent	<code>Dog</code> and <code>Cat</code> both extend <code>Animal</code>
Multiple	One class extends more than one class	<code>Manager</code> extends <code>Employee</code> and <code>Educator</code> . Not supported by Java classes (Java achieves this with interfaces, covered later)

10.2 Single inheritance [📌 GOOD TO KNOW: your slide example]

```
class Customer {                                // parent / super / base class
    String name;
    void show() { System.out.println("Customer: " + name); }
}
class RegularCustomer extends Customer {        // child / sub / derived class
    double discount = 10;
    void showDiscount() { System.out.println(name + " gets " + discount + "% discount"); }
}
public class SingleDemo {
    public static void main(String[] args) {
        RegularCustomer rc = new RegularCustomer();
        rc.name = "Ravi";                       // inherited variable
        rc.show();                               // inherited method
        rc.showDiscount();                       // own method
    }
}
```

Output:

```
Customer: Ravi
Ravi gets 10.0% discount
```

10.3 Multilevel inheritance [★ ESSENTIAL FOR EXAM: asked Feb-2022 Q3B: write a program]

```
class Animal {                                // Level 1 (grandparent)
    void eat() { System.out.println("Eating..."); }
}
class Dog extends Animal {                    // Level 2 (parent of Puppy)
    void bark() { System.out.println("Barking..."); }
}
class Puppy extends Dog {                     // Level 3 (child)
    void weep() { System.out.println("Weeping..."); }
}
public class MultilevelDemo {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat();                               // from Animal (grandparent)
        p.bark();                               // from Dog (parent)
        p.weep();                               // its own
    }
}
```

Output:

```
Eating...
Barking...
Weeping...
```

Puppy automatically gets **everything from `Dog` and from `Animal`**, like a family chain grandparent → parent → child.

10.4 Hierarchical inheritance [★ ESSENTIAL FOR EXAM: asked Jan-2023 Q4A: definition + example]

Definition: When **more than one class** inherits from the **same single parent**, it is hierarchical inheritance (one parent, many children, like a tree).

```

class Animal {
    void eat() { System.out.println("Eating..."); }
}
class Dog extends Animal {
    void bark() { System.out.println("Barking..."); }
}
class Cat extends Animal {
    void meow() { System.out.println("Meowing..."); }
}
public class HierarchicalDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat(); d.bark();
        Cat c = new Cat();
        c.eat(); c.meow();
    }
}

```

Output:

```

Eating...
Barking...
Eating...
Meowing...

```

10.5 The super keyword and constructor order [GOOD TO KNOW: in your slides (Super class)]

super refers to the **parent class**. `super(...)` calls the parent constructor and `super.method()` calls a parent method. **The parent constructor always runs first**, then the child's.

```

class Parent {
    Parent() { System.out.println("Parent constructor"); }
    void greet() { System.out.println("Hello from Parent"); }
}
class Child extends Parent {
    Child() {
        super(); // calls Parent() (implicit if
omitted)
        System.out.println("Child constructor");
    }
    void greetBoth() {
        super.greet(); // parent's method
        System.out.println("Hello from Child");
    }
}
public class SuperDemo {
    public static void main(String[] args) {
        new Child().greetBoth();
    }
}

```

Output:

```

Parent constructor
Child constructor
Hello from Parent
Hello from Child

```

COMMON MISTAKE

Constructors are not inherited, but they are always chained: creating a child object always runs the parent constructor first.

10.6 Advantages of inheritance [GOOD TO KNOW: in your slides]

- **Code reusability:** write common code once in the parent; all children use it.
- **Saves time and effort** because there is less code to write and test.
- **Flexibility:** a change in the parent automatically reaches all children.
- **Better organisation:** code is split into smaller, simpler, more readable classes that are easy to debug.

REMEMBER

Why no multiple inheritance of classes? If `C` extends both `A` and `B`, and both have a method `show()`, Java cannot decide which one to use (the **diamond problem**). So Java forbids it for classes.

Unit 11: Revision Kit [★ ESSENTIAL FOR EXAM: use in the last 24 hours]

11.1 One-page "differentiate" cheat sheet [💧 HIGH-YIELD: comparison questions are 1/3 of all questions]

Comparison	Side A	Side B
Constructor vs Method	Same name as class, no return type , automatic call, initializes object	Any name, has return type , explicit call, performs action
JDK vs JRE vs JVM	JDK = develop (javac + JRE)	JRE = run (JVM + libraries). JVM = executes bytecode
while vs do-while	Checks condition first , may run 0 times	Runs body first , runs at least once, needs ;
if-else vs else-if ladder	2 outcomes, one condition	3+ outcomes, first true block runs
switch vs else-if	One value vs constants, needs break	Any condition / range
Static vs non-static method	Class-level, no object, only static members, no this	Object-level, needs object, all members, has this
Static vs instance variable	One shared copy	One per object
== vs equals()	Compares references	Compares content
break vs continue	Exits the loop	Skips current iteration
Primitive vs non-primitive	Stores value, 8 types, lowercase	Stores reference, default null
Widening vs narrowing	Automatic, safe, small to large	Manual cast (int), may lose data
Single / multilevel / hierarchical	A>B / A>B>C / A>B,C,D	Multiple (A,B > C) is not allowed for classes

11.2 Syntax quick card [★ ESSENTIAL FOR EXAM: memorise these skeletons]

Skeletons

```

// while
while (cond) { ... }

// for
for (int i=0; i<n; i++) { ... }

// do-while
do { ... } while (cond);

// if-else-if
if (c1) {...}
else if (c2) {...}
else {...}

// switch
switch (x) { case 1: ...; break;
              default: ...; }

// array
int[] a = {1,2,3};
= 0;
a.length
... }

// constructor
ClassName(params) { this.x = x; }

// static
static int count

// object
static void m() {

ClassName o = new ClassName(args);
ClassName.m();

// inheritance
class B extends A { ... }
super(...); super.m();

// String
s.length() s.charAt(i) s.substring(b,e) s.equals(t)
s.concat(t) s.contains(t) s.replace(a,b) s.compareTo(t)

```

11.3 Practice questions with answers [🔥 HIGH-YIELD: dry-run each before checking]

Cover the answer column and work each out on paper first.

#	Code	Output	Why
1	<pre>for(int i=1;i<=3;i++) System.out.print(i*i+" ");</pre>	1 4 9	Squares of 1, 2, 3
2	<pre>int x=5; System.out.println(x++ + ++x);</pre>	12	<code>x++</code> uses 5 then <code>x=6</code> ; <code>++x</code> makes 7; 5+7
3	<pre>String s="Java"; s.concat(" Fun"); System.out.println(s);</pre>	Java	Strings are immutable; result was not stored
4	<pre>int a=7/2; double b=7/2; double c=7/2.0; print a, b, c</pre>	3 3.0 3.5	Integer division happens before assignment to b
5	<pre>int n=1; switch(n){case 1: print("One"); case 2: print("Two"); break; case 3: print("Three");}</pre>	OneTwo	No break after case 1, so falls into case 2, then break
6	<pre>class A{static int s=0; int i=0; A(){s++; i++;}} create 3 objects, print s and i of the last</pre>	3 1	<code>s</code> is shared (3 objects), <code>i</code> is per object
7	<pre>class T{ void T(){ System.out.println("hi"); } } then new T();</pre>	(nothing)	<code>void T()</code> is a method, not a constructor
8	<pre>for(i=1..3){ for(j=1..i){ print(j+" "); } println(); }</pre>	1 / 1 2 / 1 2 3 on 3 lines	Inner loop runs <code>i</code> times
9	<pre>System.out.println(10 + 20 + "Java" + 10 + 20);</pre>	30Java1020	Left to right: <code>10+20=30</code> first, then everything after a String is joined as text
10	<pre>String s="Hello"; System.out.println(s.substring(1,4) + s.length() + s.charAt(0));</pre>	ell5H	<code>substring(1,4) = "ell"</code> , <code>length=5</code> , <code>charAt(0)=H</code>
11	<pre>int i=0; while(i<3){ if(i==1){i++; continue;} System.out.println(i); i++; }</pre>	0 and 2 (two lines)	When <code>i</code> is 1 it increments and skips the print
12	<pre>char c='A'; c++; System.out.println(c);</pre>	B	<code>char</code> is stored as a number; next value is B

11.4 Top mistakes that lose marks [★ ESSENTIAL FOR EXAM: avoid in the exam]

Mistake	Fix
Giving only theory, no code	Almost every question says "with suitable example/program". Always add code and output.
Writing <code>=</code> instead of <code>==</code>	<code>=</code> assigns, <code>==</code> compares.
Using <code>==</code> for Strings	Use <code>.equals()</code> for content comparison.
Wrong substring end index	<code>substring(b,e)</code> excludes <code>e</code> .
Constructor with a return type	No <code>void</code> , no return type, same name as class.
Using <code>this</code> or instance variables inside a static method	Not allowed. Create an object first.
Forgetting <code>break</code> in switch	Causes fall-through into the next case.

Mistake	Fix
Forgetting the update in a loop	Creates an infinite loop.
Skipping the comparison table in "differentiate" questions	Always draw a table with 4 to 6 rows.
Not stating assumptions	The paper says "assume suitable data". Write "Assume HRA = 20%" etc.

11.5 Exam strategy: how to write an 8-mark answer [★ ESSENTIAL FOR EXAM: works for every question here]

1. **Definition** (1 to 2 lines, in your own simple words).
2. **Key points or comparison table** (4 to 6 rows).
3. **Code example** with proper braces and indentation.
4. **Output** written next to the code.
5. **A small diagram** (flow, memory box or hierarchy) where it fits. It earns easy marks and fills space neatly.

REMEMBER

Time rule: about 1.8 minutes per mark. An 8-mark answer should take roughly 12 to 14 minutes. If a program is long, write the **class skeleton first**, then fill in the methods.

11.6 Self-test checklist [★ ESSENTIAL FOR EXAM: tick when you can do it from memory]

✓	Can you...	Unit
☐	Draw JDK, JRE, JVM nested and write the comparison table?	1
☐	Explain write once run anywhere with bytecode and JVM?	1
☐	List the 5 token types with examples?	2
☐	Write the else-if ladder grade program?	3
☐	Write a switch program and explain fall-through?	3
☐	List all six relational operators with a program?	4
☐	Trace a loop using a table?	5
☐	Write while vs do-while with both examples?	5
☐	Print the 1 / 0 1 / 1 0 1 ... pattern?	5
☐	Use substring, equals, concat, compareTo, contains, replace, length with outputs?	6
☐	Write the Employee gross salary program?	7
☐	Write constructor vs method table plus program?	8
☐	Write static variable, static method and static block programs?	9
☐	Write multilevel and hierarchical inheritance programs?	10

Good luck! Master the five red-tagged units (8, 1, 5, 6, 9) and you have covered about two-thirds of the marks the past papers placed on your sessional content.